

Implementação de Data Warehouse com PostgreSQL

Aula 5 — Business Intelligence na Prática



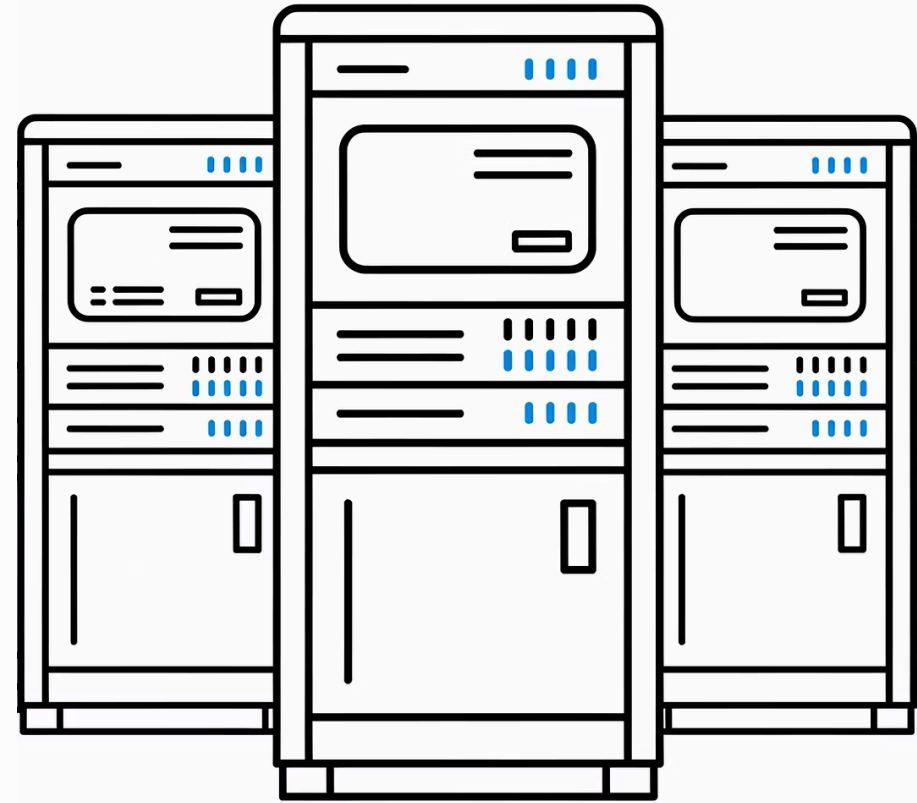
Modelagem Dimensional



PostgreSQL Aplicado



Integração com Power BI



O Que Vimos Até Agora

Nas aulas anteriores, construímos a base teórica e prática para entender o mundo do Business Intelligence. Agora chegou o momento de colocar tudo isso em prática armazenando dados em um Data Warehouse real.

Business Intelligence

Conceito e aplicações no mundo corporativo

BI, Data Science e Analytics

Diferenças e quando usar cada abordagem

Modelagem Multidimensional

Star Schema, tabelas Fato e Dimensão

Power Query

Preparação e transformação de dados

- 📌 A partir de hoje, vamos **armazenar esses dados em um Data Warehouse real** utilizando PostgreSQL como banco de dados analítico.

Por Que Usar um Data Warehouse?

Empresas de todos os portes utilizam Data Warehouses para centralizar informações dispersas em diferentes sistemas e transformá-las em inteligência estratégica.

Consolidação de Dados

Integra dados do ERP, CRM e sistemas de vendas em uma única fonte confiável de verdade.



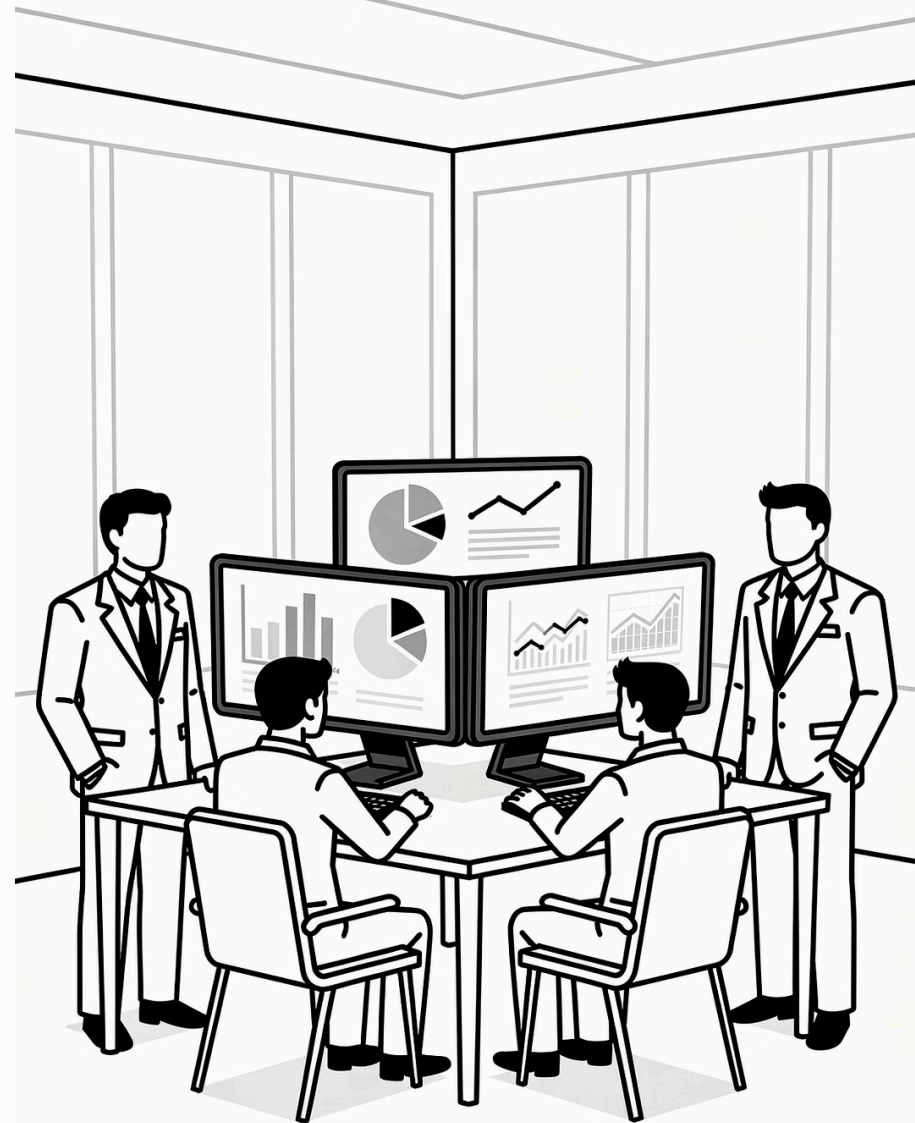
Relatórios Estratégicos

Suporta análises históricas e tendências para decisões gerenciais mais embasadas.



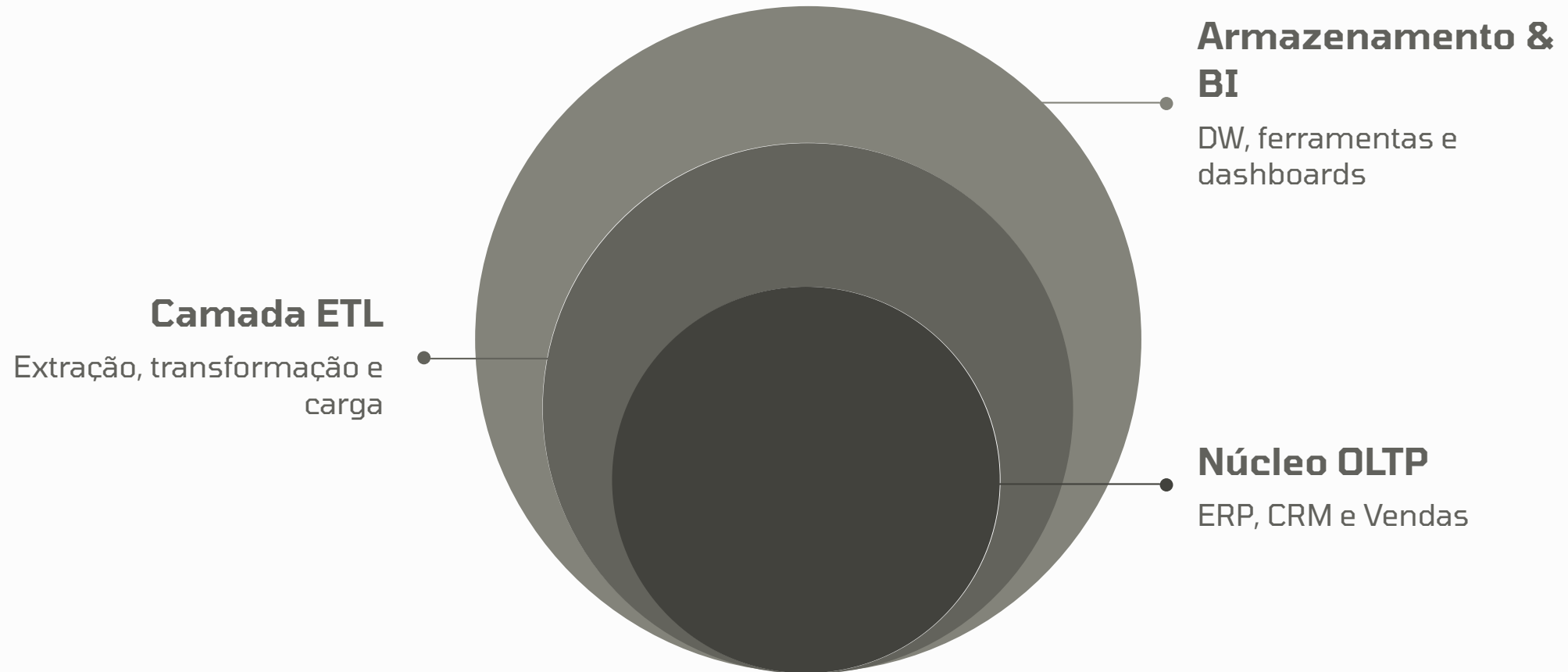
Histórico Preservado

Garante rastreabilidade temporal dos dados, permitindo comparações entre períodos.



Arquitetura Simplificada de BI

O fluxo padrão de uma solução de Business Intelligence percorre camadas, desde os sistemas transacionais até os dashboards consumidos pelos gestores.



Cada camada tem uma responsabilidade clara: os sistemas operacionais **geram** dados, o ETL os **transforma**, o Data Warehouse os **armazena** de forma analítica, e o BI os **apresenta** para tomada de decisão.

OLTP vs. OLAP

Compreender a diferença entre sistemas operacionais e analíticos é fundamental para projetar um Data Warehouse eficiente. Nosso DW será inteiramente **OLAP**.

OLTP — Operacional

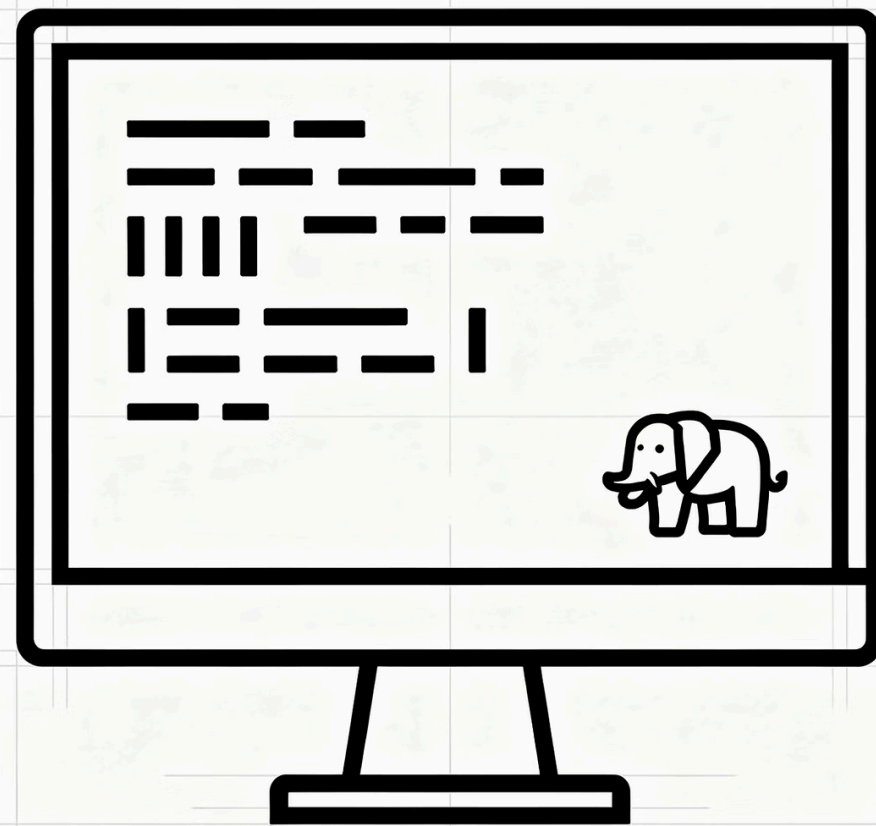
- Muitas transações simultâneas
- Dados normalizados (3FN)
- Foco em registro e atualização
- Exemplos: ERP, CRM, e-commerce

OLAP — Analítico

- Leitura intensiva e consultas complexas
- Dados agregados e desnormalizados
- Foco em análise e histórico
- Exemplos: Data Warehouse, Data Mart

O Que é PostgreSQL?

PostgreSQL é um **Sistema Gerenciador de Banco de Dados Relacional (SGBD) open source**, reconhecido mundialmente pela robustez, conformidade com padrões SQL e excelente desempenho analítico. É gratuito, extensível e amplamente adotado no mercado.



Por Que PostgreSQL em BI?

Entre os SGBDs disponíveis no mercado, o PostgreSQL se destaca como uma das melhores opções para **Data Warehouses**, especialmente em ambientes acadêmicos e empresas que buscam soluções open source de alta qualidade.

→ Gratuito e Open Source

Zero custo de licenciamento, com código aberto e comunidade global ativa.

→ SQL Avançado

Suporte completo a CTEs, window functions, particionamento e extensões analíticas.

→ Alta Performance e Escalabilidade

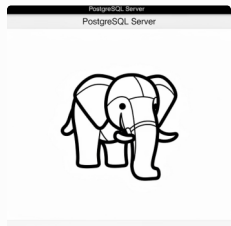
Suporta grandes volumes de dados com índices eficientes e otimizador de consultas sofisticado.

→ Compatível com Power BI

Conexão nativa via conector oficial, sem necessidade de middleware adicional.

Ferramentas da Aula

Durante esta aula prática, trabalharemos com um conjunto de ferramentas que formam um pipeline completo de dados — da fonte bruta até o dashboard analítico.



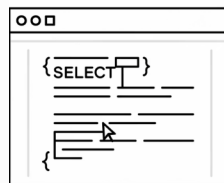
PostgreSQL

Banco de dados analítico onde o DW será construído



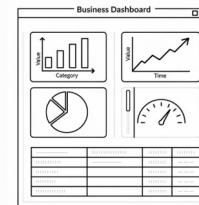
PgAdmin

Interface gráfica para gerenciar e executar SQL no PostgreSQL



SQL

Linguagem padrão para criação e manipulação de tabelas do DW



Power BI

Ferramenta de visualização conectada diretamente ao PostgreSQL

📄 **Fluxo da aula:** CSV → PostgreSQL → Power BI

Dataset: Supermercado ABC 100k

Nosso projeto utilizará uma base com **100.000 registros** de vendas de um supermercado fictício, cobrindo todas as dimensões necessárias para um DW completo.

100K

Registros

Volume total de transações de
venda na base bruta

5

Dimensões

Produto, Filial, Cliente, Pagamento e
Calendário

1

Tabela Fato

Fato_Vendas centralizando todas as
métricas de negócio

Problemas de Qualidade na Base Bruta

Dados reais raramente chegam limpos. A base do supermercado simula os problemas típicos encontrados em sistemas corporativos — exatamente o que você vai encontrar no mercado de trabalho.

Datas Inválidas

Registros com formatos inconsistentes como `32/13/2023` ou campos nulos em colunas obrigatórias.

Espaços Extras e Encoding

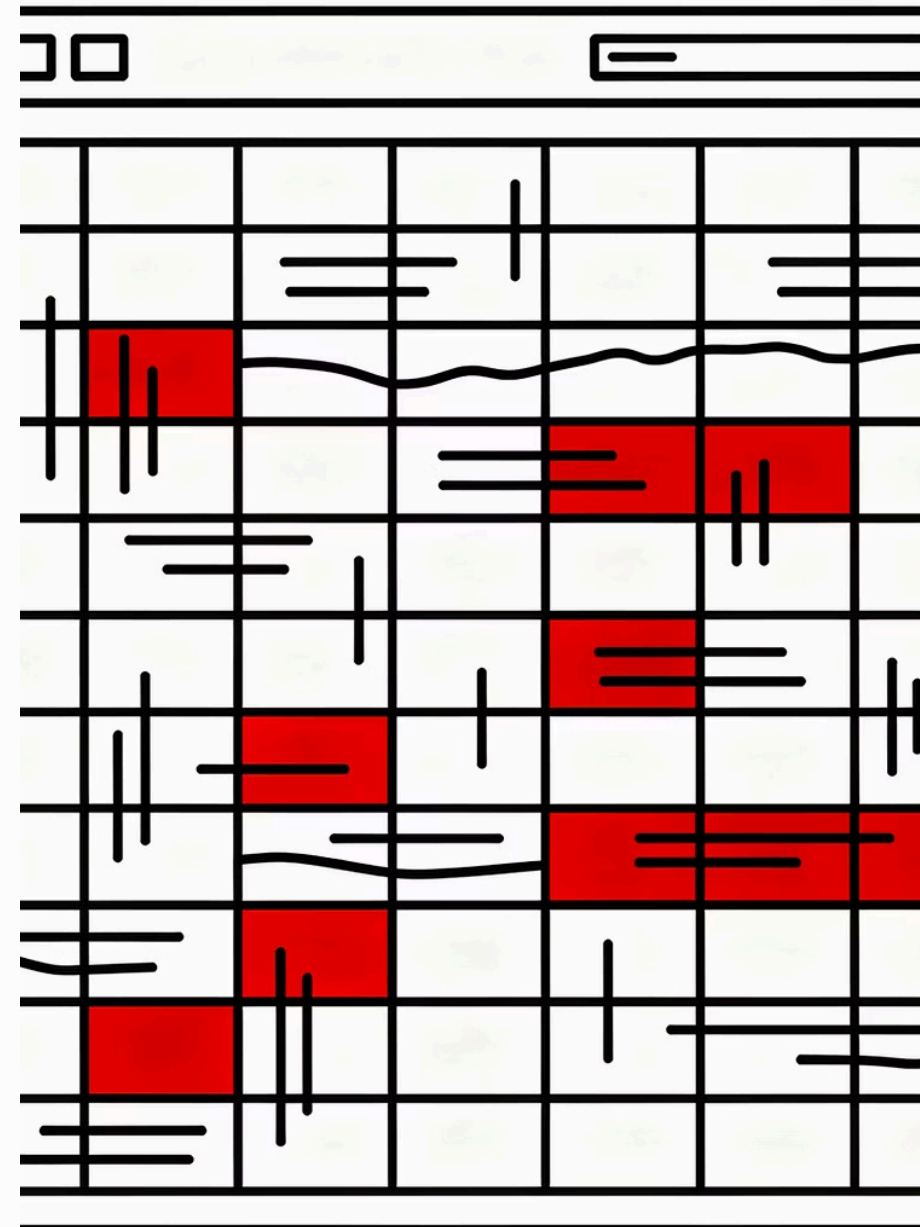
Textos com espaços no início/fim e caracteres especiais que quebram consultas SQL.

Tipos Incorretos

Valores monetários armazenados como texto (`"R$ 1.200,50"`), impossibilitando cálculos diretos.

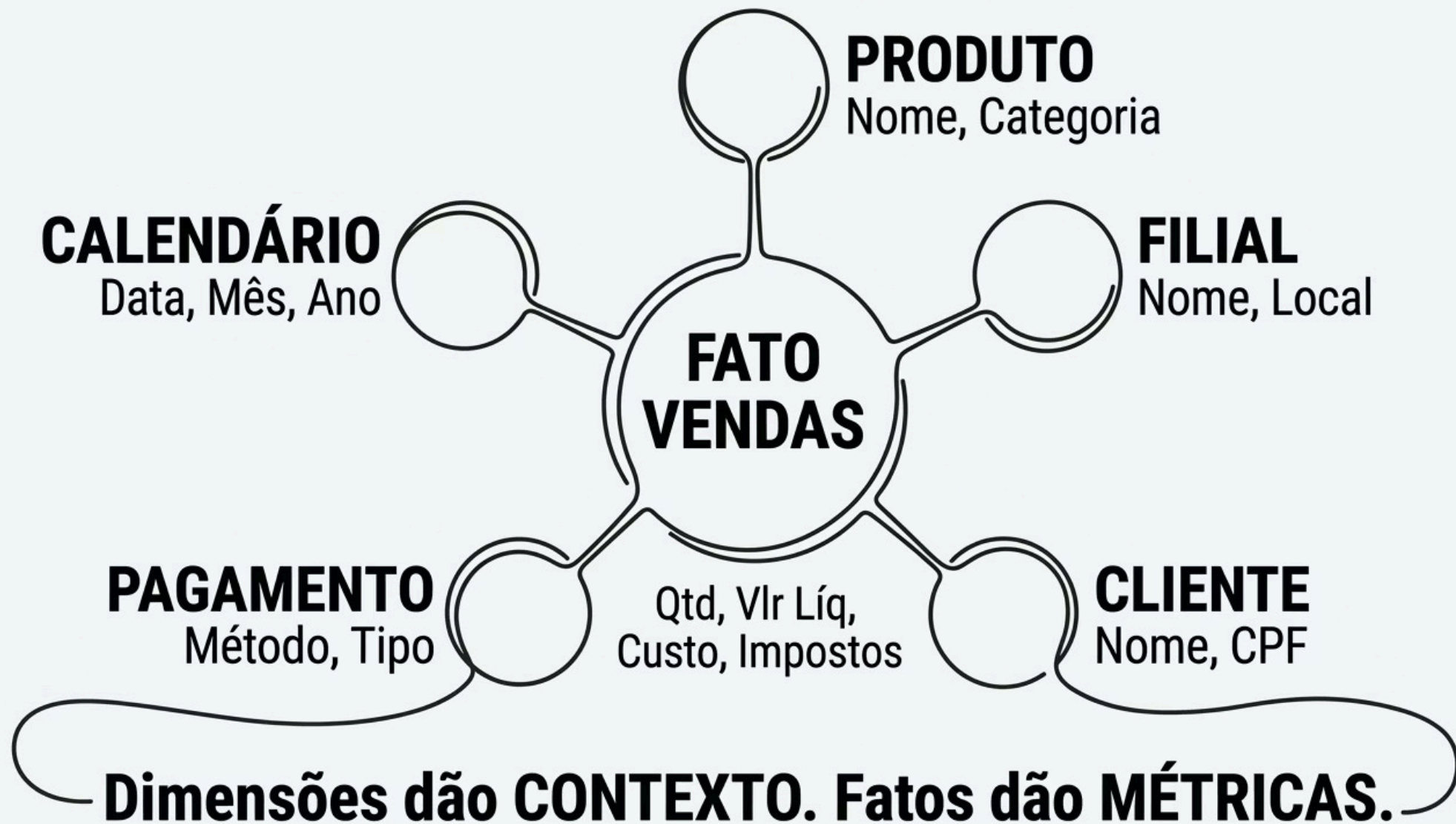
Duplicidades e Campos Vazios

Linhas duplicadas e atributos obrigatórios ausentes que comprometem a integridade referencial.



Nosso Objetivo: Um DW Dimensional

A meta é transformar a base bruta e desorganizada em um **Data Warehouse estruturado no modelo Star Schema**, com dimensões bem definidas e uma tabela fato central com as métricas de negócio.



Entendendo o Star Schema do Projeto

Tabelas Dimensão — O Contexto

As dimensões respondem às perguntas **quem, o quê, onde e quando**. Elas armazenam os atributos descritivos dos dados.

- **Dim_Produto:** produto, marca, categoria, subcategoria
- **Dim_Filial:** nome, cidade, estado
- **Dim_Cliente:** código e segmento do cliente
- **Dim_Pagamento:** forma de pagamento utilizada
- **Dim_Calendarario:** data, mês, trimestre, ano

Tabela Fato — As Métricas

A Fato_Vendas armazena as **chaves estrangeiras** das dimensões e as **medidas numéricas** do negócio:

- Quantidade vendida
- Valor líquido da venda
- Custo total do produto
- Imposto aplicado

Criando o Banco de Dados

O primeiro passo da implementação é criar o banco de dados dedicado ao Data Warehouse. No PostgreSQL, cada banco tem seu próprio espaço de armazenamento isolado.

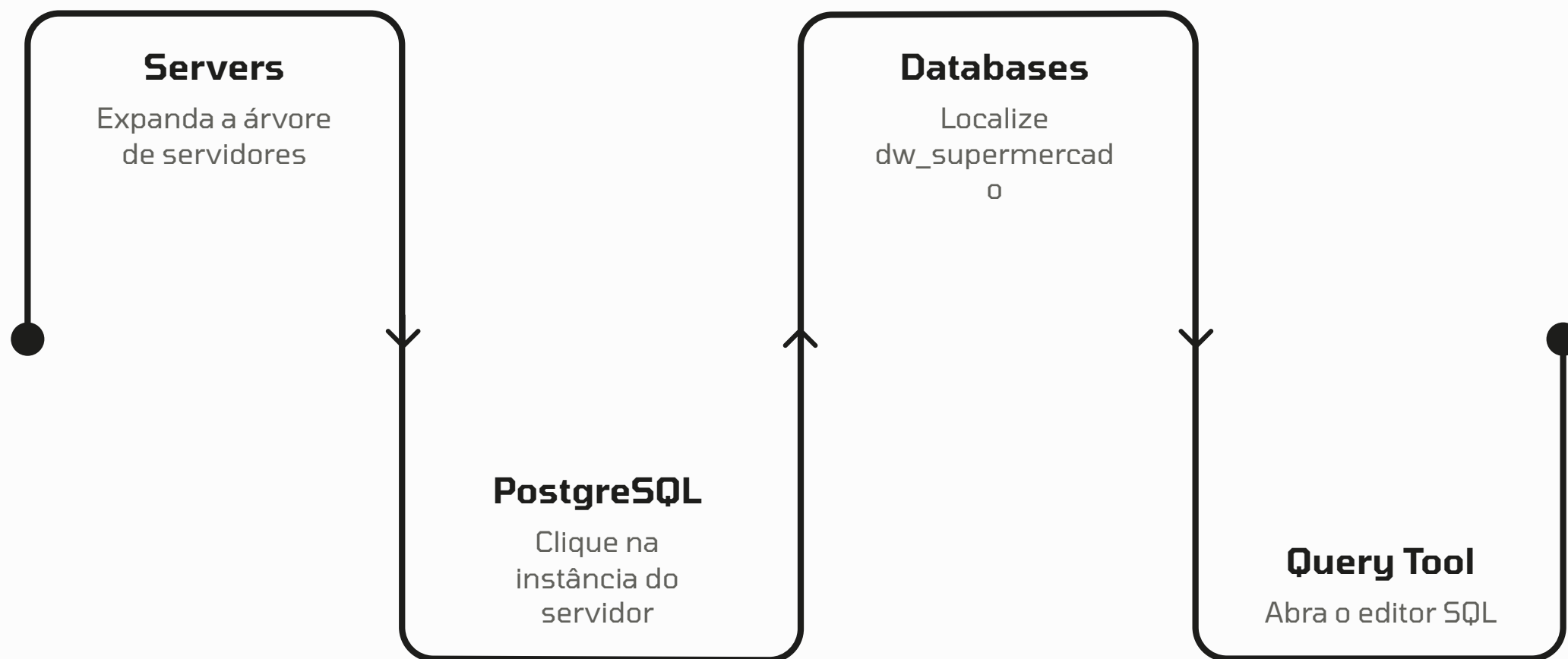
```
-- Criando o banco de dados do Data Warehouse  
CREATE DATABASE dw_supermercado;
```

📄 💡 **Dica:** No PgAdmin, você também pode criar o banco pela interface gráfica clicando com o botão direito em **Databases** → **Create** → **Database**. O comando SQL acima produz exatamente o mesmo resultado.

Após a criação, conecte-se ao banco `dw_supermercado` antes de executar os próximos comandos. Todos os objetos (schemas, tabelas) serão criados dentro deste banco.

Conectando ao Banco no PgAdmin

Com o banco criado, navegue pela árvore do PgAdmin para abrir o ambiente de execução de consultas SQL. Este será seu principal ambiente de trabalho durante a aula.



O **Query Tool** é o editor SQL do PgAdmin. Nele você escreverá e executará todos os comandos de criação das tabelas, inserção de dados e consultas analíticas do Data Warehouse.

Criando o Schema do Data Warehouse

Um **schema** no PostgreSQL funciona como um namespace — um container lógico que agrupa tabelas relacionadas. Usar um schema separado para o DW é uma boa prática que facilita a organização e o controle de acesso.

```
-- Criando o schema analítico  
CREATE SCHEMA dw;
```

Schema: public

Tabelas operacionais e de staging

Schema: dw

Dimensões e tabela fato do Data Warehouse

📌 Ao referenciar tabelas do DW, sempre use a notação `dw.nome_tabela`, como `dw.fato_vendas`.

Criar a tabela bruta no PostgreSQL

Como a planilha tem problemas intencionais, o ideal é carregar quase tudo como **TEXT**, para não quebrar na importação.

```
CREATE TABLE dw.supermercado_abc (  
  id_item      TEXT,  
  id_venda     TEXT,  
  data_hora    TEXT,  
  filial_id    TEXT,  
  filial       TEXT,  
  cidade       TEXT,  
  estado       TEXT,  
  caixa        TEXT,  
  operador_caixa TEXT,  
  id_cliente   TEXT,  
  segmento_cliente TEXT,  
  produto      TEXT,  
  categoria    TEXT,  
  subcategoria TEXT,  
  marca        TEXT,  
  quantidade   TEXT,  
  preco_unitario TEXT,  
  desconto_valor TEXT,  
  valor_liquido TEXT,  
  custo_total  TEXT,  
  imposto      TEXT,  
  comissao     TEXT,  
  forma_pagamento TEXT  
);
```


Como importar o Excel para o PostgreSQL

O PostgreSQL **não importa** `.xlsx` **diretamente via SQL puro** de forma nativa.

O mais simples é:

- abra a aba **Transacoes_RAW**
- salve como **CSV UTF-8**

Script para carregar o CSV

Se o arquivo estiver no servidor/local acessível ao PostgreSQL:

```
COPY dw.supermercado_abc  
FROM 'C:\Supermercado_ABC_100k.csv'  
DELIMITER ';'   
CSV HEADER  
ENCODING 'UTF8';
```

Consultas para validar a carga

Quantidade de linhas

```
SELECT COUNT(*)  
FROM dw.supermercado_abc;
```

Visualizar amostra

```
SELECT *  
FROM dw.supermercado_abc  
LIMIT 20;
```

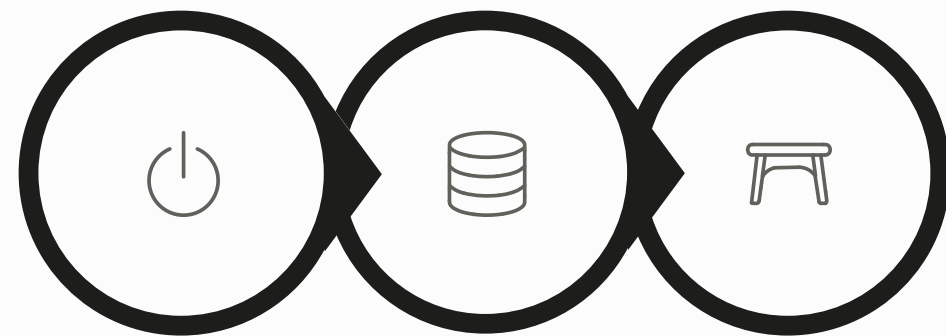
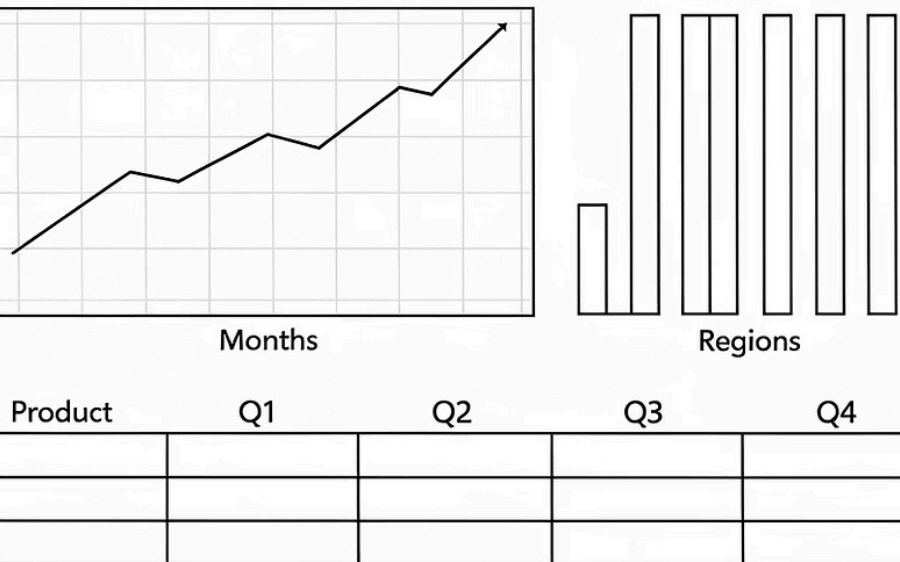
Ver possíveis valores problemáticos

```
SELECT data_hora  
FROM dw.supermercado_abc  
WHERE data_hora IS NULL  
   OR data_hora = ""  
LIMIT 20;
```

Conectar o Power BI ao PostgreSQL

Com o Data Warehouse populado, o último passo é conectar o Power BI diretamente ao PostgreSQL para criar dashboards analíticos a partir das tabelas dimensão e fato.

ales Analytics



Power BI

Configurar

Selecionar

1 Obter Dados → Banco de Dados → PostgreSQL

No Power BI Desktop, acesse **Página Inicial** e clique em **Obter Dados**.

2 Informar Servidor e Banco

Digite o endereço do servidor (ex: localhost) e o nome do banco: dw_supermercado.

3 Selecionar Tabelas do DW

Marque todas as tabelas do schema dw

Resumo da Aula 5

Nesta aula construímos do zero um Data Warehouse funcional com PostgreSQL, cobrindo todas as etapas do projeto — da modelagem à conexão com o Power BI.



Revisão Conceitual

OLTP vs OLAP, arquitetura de BI e o papel do Data Warehouse



PostgreSQL para BI

Por que usar, vantagens e ferramentas do ecossistema



Estrutura do DW

Star Schema com 5 dimensões e a Fato_Vendas



Implementação SQL

Criação de banco, schema, dimensões e tabela fato



Carga ETL

Inserção nas dimensões e população da tabela fato



Conexão Power BI

Integração do DW com a ferramenta de visualização